

MASTER DEVELOPMENT BLUEPRINT

PART-2: SECURITY & API STANDARDS

School ERP

Spring Security | JWT + Refresh Token | Redis

Prepared by: Solution Architecture / Engineering Leadership Team

Document Version: 1.0

Date: 29 July 2026

Classification: Confidential — Engineering Team Only

Preamble

This document is Part-2 of the Master Development Blueprint (MDB) v1.0 for the School ERP System, continuing directly from Part-1: Project Foundation. It defines the mandatory security, authentication, authorization, API, and validation standards that every module's implementation must follow, using the technology stack fixed in Part-1 Section 3 (React + TypeScript + MUI, Java 21 + Spring Boot 3, PostgreSQL, Redis) and now explicitly adding Spring Security and JWT + Refresh Token as the concrete realization of the authentication/authorization architecture already established in Appendix-I Section 9, Appendix-K Sections 4-5, and Appendix-F's Security NFR series.

This document does not regenerate RGD v2.0, any prior Appendix, any UI/UX Screen Specification, or MDB Part-1 — every reference below is by ID/name to those sources. It contains no source code, no API implementation, no database table definitions, no entity definitions, no JSON examples, and no sequence diagrams — every standard is stated as a rule or convention, not demonstrated in a runnable artifact.

11. Authentication Architecture

Spring Security is the mandatory authentication framework, realizing the Authentication Flow already defined in Appendix-K Section 4 and the identity model in Appendix-I Section 9.

- A dedicated Spring Security filter chain intercepts every request before it reaches a Controller, validating the JWT (Section 13) and populating the security context with the authenticated identity — this is the concrete realization of Appendix-K Section 4's "token validated at the Controller boundary" requirement.
- Credential validation on login uses Spring Security's PasswordEncoder abstraction (BCrypt or a stronger successor) against User.password_hash (Appendix-H ENT-SYS-001) — plaintext password comparison is never permitted, consistent with Appendix-F NFR-PWD-001.
- Account lockout (NFR-PWD-003) is implemented via a failed-attempt counter checked in the authentication provider, not left to ad-hoc Controller logic — five consecutive failures lock the account per Appendix-C v1.1/Appendix-F, with unlock handled by IT Admin action (Appendix-I Section 8.1).
- Multi-Factor Authentication is not enabled at initial go-live (NFR-AUTHN-002) but the Spring Security filter chain is structured with an explicit extension point for a future MFA provider, so it can be added without restructuring the authentication flow.
- The Public Zone endpoints (Admission inquiry/application submission, per Appendix-L Section 19.1) are the only endpoints excluded from the authentication filter — every other endpoint requires a valid JWT, with no exceptions granted ad hoc per Controller.

12. Authorization (RBAC)

Spring Security method-level authorization (annotation-based) is the mandatory mechanism for enforcing the Role Permission model already defined in Appendix-E Field 38 and Appendix-I Section 8.3.

- Every Service method that performs a Create/Update/Delete/Approve/Reject/Export/Print action is annotated with a role/authority check matching its Appendix-E Field 38 permission — authorization is enforced at the Service layer, not only at the Controller or route layer, consistent with Appendix-J Section 3's Business Rule enforcement principle.
- Roles map one-to-one to the Role entity (Appendix-H ENT-SYS-002); Spring Security authorities are derived from the RolePermission junction (Appendix-H Section 6) at authentication time and embedded in the JWT claims (Section 13), avoiding a database lookup on every authorization check.
- Field-level and record-level restrictions that cannot be expressed as a simple role check (e.g., a Parent seeing only their own linked children, a Teacher seeing only their assigned Section) are enforced in the Service layer's query construction, not assumed to be handled by the annotation-based check alone.
- The Finance-Team-exclusive refund authority (BR-FEE-002) and other single-role-exclusive rules are implemented as an explicit authority check separate from the general module-level permission, so that even a role with broader Fee module access cannot incidentally satisfy the refund check.
- Authorization failures return the standard Error Response (Section 15) without revealing whether the target resource exists, consistent with Appendix-K Section 8's Authorization Failure category.

13. JWT & Refresh Token Lifecycle

Stage	Standard
Access Token Issuance	Short-lived signed JWT issued on successful authentication (Section 11); claims include user identity, role, and authority set (Section 12) — no Sensitive/PII payload data beyond identity is embedded in the token
Access Token Validation	Verified on every request at the Spring Security filter chain (Section 11); signature and expiry checked before any Controller executes
Access Token Expiry	Deliberately short-lived (exact duration per NFR-SESS-001, Client/Product Decision Required); expiry forces refresh rather than indefinite validity
Refresh Token Issuance	Longer-lived, issued alongside the access token; stored server-side (Redis) to allow explicit revocation, not purely client-held
Refresh Flow	Client exchanges a valid, non-revoked refresh token for a new access token without requiring the user to re-enter credentials
Revocation	Triggered on logout, password change, and access revocation (BR-HR-002 staff exit) — revoked refresh tokens are rejected immediately, consistent with Appendix-I Section 9.2
Redis Role	Stores active/revoked refresh token state and, optionally, session-adjacent data for fast validation without a primary-database round trip on every refresh

This lifecycle is the concrete, technology-specific implementation of the conceptual Authentication Flow already defined in Appendix-K Section 4 and Appendix-I Section 9 — no new authentication behavior is introduced here beyond naming JWT and Redis as the mechanism.

14. API Design Standards

Realizes the conceptual API Design Principles already defined in Appendix-K Section 3, now fixed to a concrete REST convention.

- Every API Group catalogued in Appendix-K Section 18 is implemented as a REST resource collection, with resource-oriented, kebab-case, plural-noun paths (Part-1 Section 9 Naming Conventions) — verbs are not embedded in the path.
- Standard HTTP methods map to the Create/Read/Update/Delete/Search operations already defined per module in Appendix-K Section 18: POST for Create, GET for Read/Search, PUT/PATCH for Update, DELETE for the soft-delete operation defined in Part-1/Appendix-H.
- API versioning (Appendix-K Section 13) is implemented via a URI path segment (e.g., a leading version segment) — the specific version-increment trigger remains governed by the Backward Compatibility rules already defined in Appendix-K Section 24, not redefined here.
- Idempotency-sensitive operations (Payment recording, Admission confirmation — Appendix-K Section 14) are implemented using an idempotency-key header checked against a Redis-backed deduplication store before the underlying Service method executes.
- Rate limiting (Appendix-K Section 15, Appendix-F NFR-APIP-001) is implemented at the Spring Boot filter level, backed by Redis for distributed counting across horizontally-scaled Application Layer instances (Part-1 Section 4).

15. Request & Response Standards

Fixes the conceptual Common Request/Response Structure already defined in Appendix-K Sections 6-7 to concrete field-level conventions, without reproducing example payloads.

15.1 Request Standards

- Every authenticated request carries the JWT access token in the standard Authorization header, per Section 13.
- Every request carries a unique, client- or gateway-generated request identifier, used for audit correlation (Section 20) and idempotency (Section 14).
- Pagination, filtering, and sorting parameters (Appendix-K Sections 9-11) are passed as query parameters on GET/Search requests, using the field names already defined per entity in Appendix-G's Attribute Catalogue.

15.2 Response Standards

- Every response carries a status indicator distinguishing success, validation failure, business-rule failure, authorization failure, and system error, per Appendix-K Section 8's Error Response Standard categories.
- List/Search responses carry pagination metadata (current page, total count, page size) alongside the result set, per Appendix-K Section 9.
- Reports module responses (Appendix-K Section 18) additionally carry the Data Currency Indicator (last-refresh timestamp, provisional-data flag) already defined in Appendix-K Section 7, directly operationalizing BR-RPT-004.
- DTOs, not JPA entities, are the only object type ever serialized into a response body (Part-1 Section 10.1), preventing accidental exposure of internal persistence structure or Sensitive fields not intended for the given role.

16. Validation Standards

Implements the two-tier validation model already defined in Appendix-J Section 8 using Jakarta Bean Validation annotations on DTOs as the primary mechanism, supplemented by explicit Service-layer checks for cross-field and Business Rule validation.

Validation Type	Implementation Mechanism
Mandatory Field	Bean Validation constraint annotations on the request DTO, matching Appendix-G's Mandatory=Y attributes
Format Validation	Bean Validation pattern constraints (e.g., mobile number, Aadhaar format) matching Appendix-G's Validation Rule field
Range Validation	Bean Validation min/max constraints matching Appendix-G's attribute range rules
Uniqueness Validation	Service-layer check against the Repository before persistence, matching Appendix-H's Unique Constraints (database-level constraint is the final safeguard, not the primary check)
Cross-Field Validation	Custom Bean Validation class-level constraint or explicit Service-layer check (e.g., <code>LeaveRequest.end_date >= start_date</code>)
Business Rule Validation	Explicit Service-layer logic evaluating the specific Appendix-C v1.1 rule after Bean Validation passes — never expressed as a Bean Validation annotation alone

Client-side (React) validation mirrors these rules for user experience only and is never treated as authoritative — every rule is re-verified server-side, consistent with Appendix-J Section 8.1's Presentation-to-Application boundary principle.

17. Exception Handling Standards

Implements the five exception categories already defined in Appendix-J Section 20 using a centralized Spring @ControllerAdvice-equivalent global exception handler — no Controller or Service catches and formats its own error response independently.

Category (Appendix-J Section 20)	Handling Standard
Validation Exception	Caught centrally; returns field-level detail per Section 16, HTTP 400-class status
Business Rule Exception	Custom exception type carrying the specific Appendix-C v1.1 rule ID; caught centrally, returns the standard Business Rule Violation response (Section 15)
Integration Exception	Caught at the Event Publisher/ChannelAdapter boundary (Part-1 Section 8, administration package); never propagates to the Controller uncaught
Concurrency Exception	Raised on optimistic-locking version mismatch (Appendix-H Section 2.8, JPA @Version); caught centrally, returns the Concurrency Conflict response (Section 15)
Unhandled/System Exception	Caught by a catch-all handler; full detail logged server-side (Section 20), generic message returned to the client — no stack trace or internal identifier ever reaches the response body

18. File Upload Standards

Implements the File Upload API standards already defined in Appendix-K Section 16, using Spring's multipart request handling as the concrete mechanism.

- Every upload endpoint enforces file type allow-listing and a maximum size limit at the Spring Boot filter/interceptor level before the file reaches Service-layer logic, per Appendix-F NFR-FUS-001.
- Malware scanning occurs before the file is persisted to storage — an upload is not considered accepted, and no Document record (Appendix-G/H ENT-SYS-006) is created, until the scan passes.
- Uploaded files are stored outside the web-accessible root and are never referenced by a raw, guessable path in any API response — retrieval is always mediated through an authorized Service-layer call, per Appendix-I Section 12.3.
- Generated documents (report cards, ID cards, payslips, receipts) follow the same Document-record pattern as uploads, produced server-side by the owning module's Service layer (e.g., JasperReports for Reports/Examination/Fee outputs) rather than assembled client-side.

19. Security Standards

Fixes the Appendix-F Security NFR series and Appendix-K Section 19 API Security controls to concrete Spring Security / PostgreSQL mechanisms.

Control	Concrete Implementation
Transport Encryption	TLS enforced at the ingress/load-balancer tier (Part-1 Section 4); no endpoint reachable over plain HTTP
CSRF Protection	Spring Security CSRF tokens on all state-changing requests not authenticated purely by a bearer JWT in a context where CSRF applies
XSS Protection	All user-supplied content sanitized/escaped server-side before storage and again on render (React's default escaping is not treated as a sufficient sole control)
SQL Injection Protection	Spring Data JPA/Hibernate parameterized queries exclusively; no native/dynamic SQL string concatenation permitted anywhere in the codebase
Encryption at Rest	Field-level encryption (application-level, e.g., via a JPA attribute converter) for every Sensitive/PII column identified in Appendix-G/H's Sensitive Data Matrix
Password Storage	BCrypt (or stronger) hash only, via Spring Security's PasswordEncoder — never reversible encryption for passwords
Session/Token Security	Short-lived JWT with server-tracked refresh tokens in Redis (Section 13), not long-lived client-only tokens
Dependency Security	Maven dependency versions pinned and monitored; no dependency introduced without a known-vulnerability check

These standards are the mandatory, technology-specific floor for every module — a module implementation that does not meet every applicable row above does not pass the Security-focused Non-Functional Testing already defined in Appendix-M Section 10.

20. Logging & Audit Standards

Implements the Logging Flow and Audit Trail Processing already defined in Appendix-J Sections 10 and 16, using a structured logging framework (SLF4J/Logback or equivalent) plus an interception mechanism for audit-specific writes.

- Every Service method that changes state calls the shared AuditLogService (administration package, Part-1 Section 6) before returning — this call is implemented as a mandatory step in the method itself, not an optional cross-cutting aspect that can be silently skipped.
- Application logs (SLF4J/Logback) are structured (not free-text) and include, at minimum, the request identifier (Section 15), authenticated user, module, and outcome — enabling correlation between a specific request and its resulting log/audit entries.
- Sensitive/PII values are never written to application logs in plaintext — AuditLog old_value/new_value snapshots (Appendix-H ENT-SYS-004) redact or reference Sensitive fields rather than storing raw values, consistent with the risk already flagged in Appendix-H Section 19.
- Log levels follow a fixed convention: ERROR for unhandled/system exceptions (Section 17), WARN for business-rule violations and authorization failures, INFO for successful state-changing operations, DEBUG reserved for non-production diagnostic detail only.
- Centralized log aggregation (Part-1/Appendix-L Section 18) is the only supported destination for application logs — writing to local, unaggregated log files as the sole log destination is not permitted in any environment beyond local development.